

# Agent Run Observatory

Project kickoff summary — decisions, setup, backlog and sprint plan

|              |                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| Date         | 2 October 2026                                                                                                        |
| Repository   | <a href="https://github.com/shulginilya/ai_agent_run_observatory">github.com/shulginilya/ai_agent_run_observatory</a> |
| Board        | <a href="https://github.com/users/shulginilya/projects/3">github.com/users/shulginilya/projects/3</a>                 |
| Local folder | ~/web_projects/ai_agent_run_observatory                                                                               |
| Status       | Planned. No application code written yet. Development starts with Sprint 1.                                           |

This document records what was discussed and decided before development: why the project exists, who works on it, which options were considered, what was set up on GitHub, and the backlog and sprint plan the work will follow.

# 1. Purpose

The repository serves two purposes, in this order:

- **Training ground** for an agentic development team: Claude Code orchestrating specialised crew members (scrum master, developer, debugger, code reviewer) through a realistic project.
- **Portfolio item**: a deployed, documented web app that shows end-to-end engineering and an understanding of how AI agent systems behave.

# 2. The crew

The repository had no project-level `.claude/` folder. The crew is installed at user level in `~/.claude/skills/` as four packages. Each provides one subagent plus skills and commands.

| Member       | Subagent                    | What it does                                                                     | Commands                               |
|--------------|-----------------------------|----------------------------------------------------------------------------------|----------------------------------------|
| scrum-master | scrum-master:story-splitter | Splits oversized items into small vertical slices with acceptance criteria       | /refine, /plan-sprint, /backlog-health |
| developer    | developer:implementer       | Builds one agreed slice and reports what changed and how it was verified         | /implement, /refactor, /verify         |
| debugger     | debugger:investigator       | Tests one hypothesis about a bug's cause and returns a verdict with evidence     | /diagnose, /bisect, /fix               |
| code-review  | code-review:reviewer        | Runs one focused review pass (correctness, security, performance, accessibility) | /review, /review-pr                    |

## Working model

- **scrum-master** turns ideas into ready GitHub issues and plans sprints.
- **developer** implementers build one slice each; the orchestrator keeps the plan.
- **code-review** reviewers check each finished slice before merge, one pass per focus area.
- **debugger** investigators test competing hypotheses in parallel when something breaks.
- Used so far: the scrum-master's refinement and sprint-planning methods, and its story-splitter agent, which split the 13-point timeline epic into seven slices.

# 3. Options considered

## Project ideas

| # | Idea                                  | Summary                                                                                                                  | Portfolio value                                 |
|---|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| 1 | <b>Agent Run Observatory</b> (chosen) | Upload Claude Code session logs; see timelines, subagents, tool calls, tokens, cost and failures; optional AI summaries. | High: distinctive and on-topic for an AI agency |
| 2 | Freelance Bid Pipeline                | CRM for Upwork-style bids: postings, proposals, stages, follow-ups, win-rate analytics.                                  | Medium-high: practical, clients understand it   |
| 3 | Async Standup & Retro Board           | Team updates, live retro board, action items.                                                                            | Medium: common idea, real-time adds depth       |
| 4 | Agency Time & Invoice Tracker         | Clients, projects, time entries, PDF invoices.                                                                           | Medium: solid business-app signal               |

## Tech stacks

| Stack                                    | Components                                                                                                             | Trade-off                                                                                          |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>A. Full-stack TypeScript</b> (chosen) | Next.js 15 App Router, TypeScript, Neon Postgres + Drizzle, Auth.js, Tailwind + shadcn/ui, Vitest + Playwright, Vercel | Most in-demand skills, one language, free hosting with a live demo; Next.js has its own complexity |

| Stack                 | Components                                                                                     | Trade-off                                                   |
|-----------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| B. Python API + React | FastAPI, SQLAlchemy, Postgres, pytest; React + Vite + TanStack Query; Docker; Fly.io or Render | Strong AI/backend signal; two languages and two deployables |
| C. Edge / lean        | Hono or SvelteKit on Cloudflare Workers + D1 + Drizzle                                         | Cheap and fast; smaller ecosystem                           |
| D. Go + HTMX          | Go (chi), HTMX, SQLite, single binary                                                          | Simple and fast; weaker UI showcase                         |

**Decision:** Idea 1 on Stack A. Every stack would share CI on GitHub Actions, a case-study README and a deployed demo.

## 4. Setup completed

| Item                | Result                                                                                                                                 |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Repository rename   | ai_agency_playground → ai_agent_run_observatory on GitHub (old URL redirects); origin updated and verified                             |
| Local folder rename | ~/web_projects/ai_agency_playground → ~/web_projects/ai_agent_run_observatory                                                          |
| Git protocol        | SSH (existing id_rsa key already authenticated with GitHub)                                                                            |
| GitHub CLI          | gh 2.4.0, logged in with an OAuth token: scopes repo, project, read:org, gist                                                          |
| Labels              | type:story task epic, epic:<name>, size:<n>                                                                                            |
| Issues              | #1-#26 created (95 points excluding the epic)                                                                                          |
| Board               | Project #3 "Agent Run Observatory", linked to the repo; Status: Backlog → Ready → In progress → In review → Done; fields Size and Epic |
| Sprints             | Milestones Sprint 1-7 with goals and due dates; 25 issues assigned, epic #12 unassigned                                                |

### Lessons from setup

- Rename the remote repository first and the local folder last: the folder rename changes the session's working directory, and a failed repo rename then leaves nothing half-done.
- Fine-grained personal access tokens cannot access user-owned Projects (boards). A standard OAuth login with the project scope was needed.
- gh 2.4.0 is old: no -p flag on login and no gh project command. The board was built through the GraphQL API instead.
- In WSL, gh auth login --web cannot open a browser; open github.com/login/device manually, or install wslu and set BROWSER=wslview.
- Verify every write: one remote URL was changed before its rename was confirmed, then reverted. The repo-first order is what kept the failure contained.

## 5. Product and architecture

**Problem.** Claude Code writes every session to `~/.claude/projects/<project>/<session>.jsonl`. These logs show exactly what an agent did, but they are unreadable as raw JSON lines, so nobody looks at them.

**Product.** Upload a session log; the app parses it into a run tree and shows a timeline, subagent runs, tool calls with results and errors, token usage, estimated cost and, optionally, an AI-written summary.

### Observed log format (from a real 550 KB session)

- Entry types: `user`, `assistant`, `system`, `attachment`, `file-history-snapshot`, `last-prompt`, `mode`, `permission-mode`, `ai-title`, `queue-operation` and others, so the parser must tolerate unknown types.
- Assistant content blocks: `thinking`, `text`, `tool_use`; user content blocks: `text`, `tool_result`.
- Entries link through `uuid` / `parentUuid`; subagent entries carry `isSidechain: true`; token counts live in `message.usage`.

### Pipeline

| Stage   | Responsibility                                                                            | Issues            |
|---------|-------------------------------------------------------------------------------------------|-------------------|
| Parse   | JSONL → typed entries; bad lines skipped and reported                                     | #6                |
| Build   | Parent links, <code>tool_use</code> ↔ <code>tool_result</code> pairing, subagent grouping | #7                |
| Measure | Tokens, duration, tool counts, errors, cost                                               | #8, #22           |
| Protect | Secret redaction, optional metadata-only storage, per-user privacy                        | #10, #23          |
| Store   | Neon Postgres via Drizzle: sessions + entries                                             | #3, #9            |
| Explore | List, timeline, subagent tree, charts, AI summary                                         | #11, #13-#21, #26 |

### Design decisions

- **Privacy first.** Real logs contain prompts and code: redaction on ingest (#10), sanitized fixtures only in the repo (#5), and a demo built from fixtures (#24).
- **Parsing as pure functions** in `lib/parser`, tested against fixtures, independent of the UI and the database.
- **Vertical slices.** Every story ships something a user can see; the timeline epic was split rather than shrunk.

## 6. Backlog

26 issues, 95 points excluding epic #12. Sizes use the 1, 2, 3, 5, 8 scale; 13 means split, never commit. Each issue has a value statement, observable acceptance criteria, an out-of-scope section and named dependencies; all passed the scrum-master readiness check apart from dependencies that close as earlier work lands.

| #  | Title                                                                               | Epic       | Type  | Pts | Depends on                   | Sprint |
|----|-------------------------------------------------------------------------------------|------------|-------|-----|------------------------------|--------|
| 1  | Scaffold Next.js 15 app with TypeScript, Tailwind, shadcn/ui and Vitest             | Foundation | task  | 3   | none                         | 1      |
| 2  | CI pipeline: lint, typecheck and test on every pull request                         | Foundation | task  | 2   | #1                           | 1      |
| 3  | Database: Neon Postgres with Drizzle schema and migrations for sessions and entries | Foundation | task  | 3   | #1                           | 1      |
| 4  | Deploy to Vercel with a preview deployment for every pull request                   | Foundation | task  | 2   | #1, #3                       | 2      |
| 5  | Sanitized sample session fixtures for tests and the demo                            | Foundation | task  | 2   | none                         | 1      |
| 6  | Parse a session .jsonl file into typed entries                                      | Ingest     | story | 5   | #1, #5                       | 1      |
| 7  | Build the run tree: parent links, tool call/result pairing and subagent runs        | Ingest     | story | 5   | #6                           | 2      |
| 8  | Compute per-session metrics: tokens, duration, tool calls and errors                | Ingest     | story | 3   | #7                           | 2      |
| 9  | Upload a session file and store it                                                  | Ingest     | story | 5   | #3, #6, #7, #8               | 2      |
| 10 | Redact secrets and private content when a session is ingested                       | Ingest     | story | 5   | #9                           | 4      |
| 11 | Sessions list page                                                                  | Explore    | story | 3   | #9                           | 3      |
| 12 | Session timeline view                                                               | Explore    | epic  | —   | #7, #8                       | —      |
| 13 | Timeline of conversation turns                                                      | Explore    | story | 3   | #7, #8                       | 3      |
| 14 | Tool call rows with expandable input and output                                     | Explore    | story | 5   | #13                          | 3      |
| 15 | Markdown replies and collapsible thinking                                           | Explore    | story | 3   | #13                          | 3      |
| 16 | Subagent runs grouped under the spawning call                                       | Explore    | story | 5   | #14                          | 5      |
| 17 | Timeline filters                                                                    | Explore    | story | 3   | #15, #16                     | 5      |
| 18 | Virtualized timeline for large sessions                                             | Explore    | story | 5   | #17                          | 6      |
| 19 | Deep links, keyboard navigation, and jump-to-next-error                             | Explore    | story | 5   | #18                          | 6      |
| 20 | Subagent run tree view                                                              | Explore    | story | 5   | #7, the first timeline slice | 5      |
| 21 | Charts: token usage over time and tool-usage breakdown                              | Explore    | story | 5   | #8                           | 6      |
| 22 | Estimated cost per session by model                                                 | Explore    | story | 3   | #8                           | 5      |
| 23 | Sign in with GitHub and keep each user's sessions private                           | Access     | story | 5   | #3, #9                       | 4      |
| 24 | Public demo mode with seeded sample sessions                                        | Access     | story | 3   | #5, #23, #11                 | 4      |
| 25 | README case study with screenshots and architecture diagram                         | Access     | task  | 2   | #4, #24                      | 7      |

| #  | Title                                            | Epic       | Type  | Pts | Depends on  | Sprint |
|----|--------------------------------------------------|------------|-------|-----|-------------|--------|
| 26 | AI summary of a session run using the Claude API | AI stretch | story | 5   | #7, #8, #23 | 7      |

## 7. Sprint plan

**Assumptions:** 1-week sprints from Monday 5 October 2026; about 20 points of capacity per sprint, about 16 committed (20% kept unallocated). These are guesses until three sprints of real velocity exist. One owner, the orchestrator, runs dependency chains inside a sprint in order.

### Sprint 1 — Oct 5-9 · 15 points

**Goal:** A CI-checked app skeleton that turns a real session file into typed entries

| # | Issue                                                                               | Pts |
|---|-------------------------------------------------------------------------------------|-----|
| 1 | Scaffold Next.js 15 app with TypeScript, Tailwind, shadcn/ui and Vitest             | 3   |
| 2 | CI pipeline: lint, typecheck and test on every pull request                         | 2   |
| 3 | Database: Neon Postgres with Drizzle schema and migrations for sessions and entries | 3   |
| 5 | Sanitized sample session fixtures for tests and the demo                            | 2   |
| 6 | Parse a session .jsonl file into typed entries                                      | 5   |

### Sprint 2 — Oct 12-16 · 15 points

**Goal:** Upload a session and see it stored, on a live URL

| # | Issue                                                                        | Pts |
|---|------------------------------------------------------------------------------|-----|
| 4 | Deploy to Vercel with a preview deployment for every pull request            | 2   |
| 7 | Build the run tree: parent links, tool call/result pairing and subagent runs | 5   |
| 8 | Compute per-session metrics: tokens, duration, tool calls and errors         | 3   |
| 9 | Upload a session file and store it                                           | 5   |

### Sprint 3 — Oct 19-23 · 14 points

**Goal:** Browse sessions and read a session's conversation and tool calls

| #  | Issue                                           | Pts |
|----|-------------------------------------------------|-----|
| 11 | Sessions list page                              | 3   |
| 13 | Timeline of conversation turns                  | 3   |
| 14 | Tool call rows with expandable input and output | 5   |
| 15 | Markdown replies and collapsible thinking       | 3   |

### Sprint 4 — Oct 26-30 · 13 points

**Goal:** Safe for real logs and showable: redaction, private accounts, public demo

| #  | Issue                                                         | Pts |
|----|---------------------------------------------------------------|-----|
| 10 | Redact secrets and private content when a session is ingested | 5   |
| 23 | Sign in with GitHub and keep each user's sessions private     | 5   |
| 24 | Public demo mode with seeded sample sessions                  | 3   |

## Sprint 5 — Nov 2-6 · 16 points

**Goal:** Understand how a multi-agent run unfolded and what it cost

| #  | Issue                                         | Pts |
|----|-----------------------------------------------|-----|
| 16 | Subagent runs grouped under the spawning call | 5   |
| 17 | Timeline filters                              | 3   |
| 20 | Subagent run tree view                        | 5   |
| 22 | Estimated cost per session by model           | 3   |

## Sprint 6 — Nov 9-13 · 15 points

**Goal:** Big sessions stay fast and navigable, with charts

| #  | Issue                                                   | Pts |
|----|---------------------------------------------------------|-----|
| 18 | Virtualized timeline for large sessions                 | 5   |
| 19 | Deep links, keyboard navigation, and jump-to-next-error | 5   |
| 21 | Charts: token usage over time and tool-usage breakdown  | 5   |

## Sprint 7 — Nov 16-20 · 7 points

**Goal:** Portfolio launch: AI summary and README case study

| #  | Issue                                                       | Pts |
|----|-------------------------------------------------------------|-----|
| 26 | AI summary of a session run using the Claude API            | 5   |
| 25 | README case study with screenshots and architecture diagram | 2   |

## Risks and caveats

- Only #1 and #5 are ready today; the rest become ready as their dependencies close, so readiness is rechecked at each sprint's planning.
- Sprints 1-3 fail the "goal survives losing the largest item" test because their goals end a dependency chain (#1 → #6; #7 → #8 → #9; #13 → #14).
- Sprint 2: if the run tree (#7) overruns, upload (#9) moves to Sprint 3 and the goal falls back to "stored via script, on a live URL".
- Sprint 7 is deliberately light (7 points) to absorb spill-over and hardening before launch.

## 8. Before development starts

- Add the workflow scope so the CI file in #2 can be pushed: `gh auth refresh -h github.com -s workflow`.
- Optional: make board #3 public for the portfolio, and add a Board view grouped by Status (the API cannot create views).
- Optional: upgrade gh from 2.4.0 to the current release from GitHub's apt repository.
- Start Sprint 1 with #1 (scaffold) and #5 (fixtures), the two items that are ready now.

## Appendix — issue details

Value statement and acceptance criteria for each issue, as filed on GitHub.

### #1 · Scaffold Next.js 15 app with TypeScript, Tailwind, shadcn/ui and Vitest

As a developer on this project, I want a working app skeleton with the agreed tooling so that every later story starts from the same conventions.

- Given a fresh clone, when I run `pnpm install && pnpm dev`, then the app serves a placeholder home page on `localhost:3000`
- Given the repo, when I run `pnpm lint`, `pnpm typecheck` and `pnpm test`, then each exits 0 (one sample Vitest test exists)
- Given a TypeScript error or lint violation is introduced, when I run the matching script, then it exits non-zero
- Tailwind and shadcn/ui are configured, and one shadcn component renders on the home page in light and dark mode

### #2 · CI pipeline: lint, typecheck and test on every pull request

As a maintainer, I want every pull request checked automatically so that broken code cannot reach main unnoticed.

- Given a PR is opened against main, when CI runs, then GitHub Actions runs lint, typecheck and unit tests and reports a status check
- Given a PR with a failing test, when CI runs, then the check is red and names the failing step
- Given no source changes since the last run, when CI runs, then the pnpm cache is restored (the run log shows a cache hit)

### #3 · Database: Neon Postgres with Drizzle schema and migrations for sessions and entries

As a developer, I want a versioned database schema so that parsed sessions can be stored and queried consistently in every environment.

- Given `DATABASE_URL` is set, when I run `pnpm db:migrate`, then tables `sessions` and `entries` exist (entries keep the raw JSON plus indexed columns: `uuid`, `parent_uuid`, `type`, `timestamp`, `is_sidechain`)
- Given `DATABASE_URL` is missing, when the app or migration starts, then it fails fast with a clear error naming the variable
- Given the migrations already ran, when I run `pnpm db:migrate` again, then nothing changes and it exits 0
- A `.env.example` documents every required variable

### #4 · Deploy to Vercel with a preview deployment for every pull request

As the portfolio owner, I want the app live on a public URL with previews per PR so that reviewers and recruiters can see working software, not just code.

- Given a merge to main, when Vercel builds, then the production URL serves the latest version
- Given a PR is opened, when Vercel builds, then a preview URL is posted on the PR
- Given the build fails, when Vercel runs, then main's production deployment stays on the previous good version
- Production and preview each use their own database branch on Neon

## #5 · Sanitized sample session fixtures for tests and the demo

As a developer, I want realistic but sanitized session logs in the repo so that tests and the public demo never contain real private conversations.

- Given the repo, when I look in `fixtures/sessions/`, then there are at least 3 files: a small session, a session with subagents, and one with failed tool calls
- Given any fixture, when it is grepped for the owner's home path, email or API-key patterns, then nothing matches (a test enforces this)
- One fixture contains a malformed line and an unknown entry type, for parser edge-case tests

## #6 · Parse a session .jsonl file into typed entries

As a developer, I want a session file turned into typed, validated entries so that every feature works from one trustworthy model instead of raw JSON.

- Given a valid session file, when it is parsed, then every user, assistant and system entry becomes a typed entry with its content blocks (text, thinking, tool\_use, tool\_result)
- Given a line that is not valid JSON, when it is parsed, then that line is skipped and reported with its line number, and the rest still parses
- Given an entry type the parser does not know (e.g. queue-operation), when it is parsed, then it is kept as a generic entry instead of failing
- Given an empty file, when it is parsed, then the result is an empty session with a "no entries" warning
- Parsing is a pure function in `lib/parser`, covered by Vitest tests against the #5 fixtures

## #7 · Build the run tree: parent links, tool call/result pairing and subagent runs

As a developer reviewing an agent run, I want entries linked into a tree with tool calls matched to their results so that the app can show cause and effect, not just a flat list.

- Given parsed entries, when the tree is built, then each entry is attached to its parent via `parentUuid`, ordered by timestamp
- Given a `tool_use` block, when the tree is built, then it is paired with the `tool_result` that has the same id, along with that result's error flag and its duration
- Given a `tool_use` with no matching result (interrupted session), when the tree is built, then it is marked "no result" instead of failing
- Given entries with `isSidechain: true`, when the tree is built, then they are grouped as a subagent run under the call that spawned them

## #8 · Compute per-session metrics: tokens, duration, tool calls and errors

As a developer reviewing agent runs, I want summary numbers for each session so that I can spot expensive or failing runs at a glance.

- Given a parsed session, when metrics are computed, then input, output and cache tokens are summed from `message.usage`, and duration is the time from the first to the last timestamp
- Given a session, when metrics are computed, then tool calls are counted per tool name, and failed calls are counted separately
- Given entries without `usage` data, when metrics are computed, then their tokens count as 0 and the session is flagged "partial token data"

## #9 · Upload a session file and store it

As a developer, I want to upload a session log from the browser so that I can explore it without running any scripts.

- Given I am on the upload page, when I drop or pick a `.jsonl` file, then it is parsed, stored with its metrics, and I am redirected to its session page
- Given a file larger than 20 MB or one that is not `.jsonl`, when I upload it, then I see a clear error and nothing is stored
- Given a file with malformed lines, when I upload it, then the session is stored and the page shows how many lines were skipped
- Given the same file is uploaded twice, when the second upload finishes, then I get the existing session, not a duplicate (matched by `sessionId`)

## #10 · Redact secrets and private content when a session is ingested

As a user uploading my session logs, I want secrets and optionally my conversation text stripped before storage so that I can use the tool without leaking credentials or private work.

- Given content containing API-key, token or private-key patterns, when it is ingested, then those values are replaced with `[REDACTED:<kind>]` before anything is stored
- Given I tick "metadata only" on upload, when it is ingested, then text and thinking content are dropped, and only the structure, tool names, timings and token counts are stored
- Given a value that only looks like a key (e.g. a UUID), when it is ingested, then it is not redacted (a test covers the false-positive cases)
- The home-directory path is replaced with `~`

## #11 · Sessions list page

As a developer, I want to see all my uploaded sessions in one table so that I can find the run I care about quickly.

- Given stored sessions, when I open `/sessions`, then I see a table with the title, project, start time, duration, tokens, tool-call count and error count, newest first
- Given I click a column header, when the table re-sorts, then the order follows that column, and the sort is kept in the URL
- Given no sessions, when I open `/sessions`, then I see an empty state with a link to upload
- Given more than 50 sessions, when I scroll, then the list is paginated

## #12 · Session timeline view (epic: split into slices by story-splitter)

As a developer reviewing an agent run, I want a chronological timeline of everything the agent did in a session so that I can understand how it reached its result and where it went wrong.

- Slices #13-#19 are all closed

### #13 · Timeline of conversation turns

As a developer reviewing an agent run, I want a chronological list of my prompts and the assistant's text replies on the session page so that I can read the conversation in order.

- Given a session with user and assistant text entries, when I open `/sessions/[id]`, then a timeline below the header lists them in timestamp order, and each turn shows a role label (User or Assistant) and a time
- Given `isMeta` entries and non-conversation types (`system`, `attachment`, `file-history-snapshot`, `last-prompt`, `mode`, `permission-mode`, `ai-title`, `queue-operation`, `unknown`), when the timeline renders, then none of them appear
- Given a session with no visible turns (an empty file or only noise entries), when I open it, then I see an empty state ("No conversation turns in this session") and no error
- Given a user entry whose content is only `tool_result` blocks, when the timeline renders, then it is not shown as a user prompt
- Text is rendered as plain text with whitespace preserved; any HTML in it is escaped, not interpreted

### #14 · Tool call rows with expandable input and output

As a developer reviewing an agent run, I want each tool call shown as a compact row that expands to its full input and result so that I can see what the agent did and spot failures.

- Given a `tool_use` paired to a `tool_result`, when the timeline renders, then a row shows the tool name, a truncated one-line input summary, the duration (result time minus call time) and an "ok" badge
- Given a paired result flagged as an error, when the row renders, then it shows an "error" badge and a highlighted background, and expanding it shows the error output
- Given a row, when I click it, then it expands to show the full input (formatted JSON) and the output, and clicking again collapses it; rows start collapsed
- Given a `tool_use` with no matching result, when it renders, then the row shows a "no result" badge and no duration, without crashing
- Given tool output over 100 KB, when the row is expanded, then the output is truncated and a "show full output" control appears
- Given parallel tool calls in one assistant message, when they render, then each gets its own row, in the order of the blocks

### #15 · Markdown replies and collapsible thinking

As a developer reviewing an agent run, I want assistant replies rendered as markdown and thinking shown in collapsible blocks so that long reasoning does not bury the actions.

- Given an assistant text block with markdown (headings, lists, code fences, links, tables), when it renders, then it displays as formatted markdown, with code fences styled as code
- Given a `thinking` block, when the timeline renders, then it appears collapsed, labelled "Thinking" with a short preview, and clicking expands the full text
- Given markdown containing raw HTML or a `javascript:` link, when it renders, then the HTML is not executed and the link is neutralized
- Given an empty or whitespace-only text or thinking block, when it renders, then no empty block appears
- Given an unterminated code fence, when it renders, then the page still renders without error

## #16 · Subagent runs grouped under the spawning call

As a developer reviewing an agent run, I want subagent (sidechain) activity nested and collapsible under the tool call that spawned it so that the main flow stays readable.

- Given a spawning tool call with sidechain entries in the run tree, when it renders, then the row shows a "subagent" indicator and the entry count, with the nested entries indented beneath it
- Given a subagent group, when I click its toggle, then it collapses or expands independently of tool-row expansion; groups start collapsed
- Given a failed tool call inside a collapsed group, when the group is rendered, then the spawning row shows an "N errors inside" marker
- Given sidechain entries that cannot be linked to a spawning call, when they render, then they appear in an "Unattached subagent activity" group at the end instead of being dropped or causing a crash
- Given a subagent that spawns another subagent, when they render, then they nest to the depth present in the data

## #17 · Timeline filters

As a developer reviewing an agent run, I want to filter the timeline by tool name, show only errors and hide thinking so that I can focus on what matters.

- Given the filter bar, when I pick tool names from the list of tools used in this session, then only rows for those tools are shown (plus their parent turns)
- Given "errors only" is on, when the timeline renders, then only failed tool calls are shown, including those inside subagent groups along with the groups that contain them
- Given "hide thinking" is on, when the timeline renders, then no thinking blocks appear
- Given filters that match nothing, when they are applied, then a "No entries match these filters" empty state appears with a "Clear filters" action
- Filter state lives in URL query params, so reloading or sharing the link keeps the view; an unknown tool name in the URL is ignored

## #18 · Virtualized timeline for large sessions

As a developer reviewing a heavy agent run, I want the timeline to scroll smoothly with thousands of entries so that big sessions stay usable.

- Given a generated fixture with 5,000+ entries, when I open its page, then the timeline renders fewer than 150 entry rows in the DOM (asserted in Playwright), and the first rows appear within 2 s of navigation on a production build
- Given variable-height rows (expanded tool rows, open groups, thinking blocks), when I expand or collapse them, then the list re-measures and scrolling does not jump and rows do not overlap
- Given a collapsed subagent group, when it is expanded, then its children are inserted and the scroll position stays anchored
- Given a session of a few hundred entries, when it renders, then the behaviour is identical to before this change
- Given a very large output in an expanded row, when I scroll past it, then the page stays responsive

## #19 · Deep links, keyboard navigation, and jump-to-next-error

As a developer reviewing an agent run, I want to link to a specific entry, move through entries with j/k and jump to the next error so that I can navigate and share findings quickly.

- Given a URL hash #entry-<uuid>, when I open the page, then the timeline scrolls to that entry, highlights it and expands any collapsed groups above it
- Given a focused entry, when I use "copy link", then the URL hash updates, and the copied link reopens at the same entry
- Given the timeline has focus, when I press j or k, then focus moves to the next or previous visible row and scrolls it into view; the keys are ignored while I'm typing in an input
- Given errors in the session, when I press "Next error", then the view jumps to the next failed tool call, expanding its groups; after the last error it shows a message instead of moving
- Given a hash for a missing or filtered-out entry, when the page loads, then a notice "Entry not found or hidden by filters" appears with a "Clear filters" option
- Given a session with no errors, when the page loads, then "Next error" is disabled

## #20 · Subagent run tree view

As a developer orchestrating several agents, I want to see which subagents a session spawned and what each one did so that I can judge whether delegating paid off.

- Given a session with subagent runs, when I open its "Agents" tab, then I see a tree: main agent → each subagent with its type, duration, tokens and result status
- Given I click a subagent, when it opens, then I see that subagent's own timeline
- Given a session with no subagents, when I open the tab, then it says so instead of showing an empty tree

## #21 · Charts: token usage over time and tool-usage breakdown

As a developer, I want charts of how a session spent tokens and tools so that I can see where the cost and time went.

- Given a session, when I open its "Insights" tab, then I see cumulative tokens over time (input, output and cache as separate series) and a bar chart of calls per tool, with failures marked
- Given I hover over a point or bar, when the tooltip shows, then it lists the exact values
- Given a session flagged "partial token data", when the chart renders, then a note says the totals are incomplete
- Charts are readable in both light and dark mode

## #22 · Estimated cost per session by model

As a developer paying for agent runs, I want an estimated dollar cost per session so that I can compare how expensive different ways of working are.

- Given a session, when metrics show, then the cost is computed from each message's model and its token counts, using a pricing table in lib/pricing
- Given a model that is not in the pricing table, when the cost is computed, then those tokens are shown as "unpriced" instead of \$0
- The UI labels the value "estimate", and the pricing table records its source and date

### **#23 · Sign in with GitHub and keep each user's sessions private**

As a user, I want to sign in with GitHub and have my sessions visible only to me so that I can upload real logs safely.

- Given I am signed out, when I click "Sign in with GitHub", then Auth.js signs me in and I land on /sessions
- Given I am signed in, when I upload a session, then it is owned by me, and no other user can open its URL (they get a 404)
- Given I am signed out, when I open an upload or private session URL, then I am redirected to sign-in
- Given I sign out, when I go back in the browser history, then private pages do not render cached data

### **#24 · Public demo mode with seeded sample sessions**

As a recruiter or client visiting the portfolio, I want to explore sample sessions without signing in so that I can judge the app in under a minute.

- Given I am signed out, when I open the home page, then a "View demo" button opens the sanitized #5 sessions read-only
- Given I am in demo mode, when I try to upload or delete, then the action is disabled and invites me to sign in
- Given a fresh deployment, when the seed script runs, then the demo sessions exist exactly once (running it again changes nothing)

### **#25 · README case study with screenshots and architecture diagram**

As the portfolio owner, I want the README to explain the project like a case study so that a reviewer understands the problem, the design and the results without running anything.

- Given the README, when a reader opens it, then it covers the problem, the live demo link, screenshots of the list, timeline and insights pages, an architecture diagram, the stack, and local setup steps
- Given a fresh clone, when the setup steps are followed exactly, then the app runs locally
- Every screenshot is taken from sanitized demo data

### **#26 · AI summary of a session run using the Claude API**

As a developer reviewing a long run, I want a short AI-written summary of what the agent tried, what worked and what failed so that I don't have to read hundreds of entries.

- Given a session, when I click "Summarize", then the server sends a condensed run outline (not raw content) to the Claude API, and the page shows the summary with goal, steps, failures and the outcome
- Given the summary already exists, when I open the session again, then the cached summary shows without a new API call
- Given the API fails or the key is missing, when I click "Summarize", then I see a clear error and the rest of the page still works
- Given a session stored "metadata only" (#10), when it is summarized, then only structure is sent, and the UI says so